

Vehicle Service Booking System

Table of Contents

Vehicle Service Booking System (MERN Stack) - Project Documentation 3

1.1 Introduction 3

1.2 Problem Statement 3

1.3 Objectives 3

1.4 Scope 3

1.5 System Architecture 3

1.6 Functional Requirements 4

1.7 Non-Functional Requirements 4

1.8 Database Design 4

1.9 API Endpoints 5

1.10 Work Breakdown Structure (WBS) 5

1.11 UI Wireframes 6

1.12 Vehicle Service Booking System (React + Node.js + MongoDB) 7

Vehicle Service Booking System (MERN Stack) – Final Project

1.1 Introduction

This project is a Vehicle Service Booking System built using the MERN Stack. The system enables customers to schedule vehicle service appointments while allowing administrators to manage, approve, and track service bookings. It is designed for a 1-month internship duration

1.2 Problem Statement

Most vehicle service centers rely on manual appointment processes, leading to booking conflicts, customer delays, and missing records. A digital booking platform solves these issues by providing structured appointment management and streamlined communication between customers and service providers.

1.3 Objectives

- Allow customers to book service appointments online
- Provide an admin dashboard for booking management
- Enable admins to approve, reject, or complete bookings
- Manage service types/categories
- Improve overall service center efficiency and workflow

1.4 Scope

The system includes:

- Customer booking module
- Admin login & authentication
- Booking approval & status management
- Service category management
- Dashboard overview

Outside the basic scope: billing, SMS/email notifications, inventory management.

1.5 System Architecture

The MERN architecture consists of:

• Frontend: React.js for UI

• Backend: Node.js + Express API

• Database: MongoDB

• Auth: JWT-based secure login

The frontend communicates with backend APIs, which interact with MongoDB to store and manage data.

1.6 Functional Requirements

• Customer can submit booking requests

• Admin can log in securely

• Admin can view all bookings

• Admin can update the status (Pending/Approved/Completed/Rejected)

• Admin can manage service types

• Users can view updated booking information

1.7 Non-Functional Requirements

• Responsive user interface

• High performance and fast response

• Secure admin login

• Scalable database

• User-friendly navigation

1.8 Database Design

Collections:

1. Users (Admin)

2. Bookings

3. ServiceCategories

Booking Document Example:

{

customerName: String, phone: String, vehicleNumber: String, serviceType: String, date: String, time: String, status: "Pending" | "Approved" | "Completed" | "Rejected", createdAt: Date

}

1.9 API Endpoints

Auth

• POST /api/auth/login

Bookings

• POST /api/bookings

• GET /api/bookings

• GET /api/bookings/:id

• PUT /api/bookings/:id/status

• DELETE /api/bookings/:id

Service Categories

• GET /api/services

• POST /api/services

1.10 Work Breakdown Structure (WBS)

Frontend

• Design UI pages

• Build booking form

• Build dashboard

• Implement API calls

• Create authentication flow

Backend

• Setup server

• Create booking APIs

• Create service category APIs

• Admin login authentication

• Integrate MongoDB

Testing & Deployment

• Test APIs with Postman

• UI testing

• Fix issues

• Deploy frontend (Vercel)

• Deploy backend (Render)

1.11 UI Wireframes

Customer Booking Page:

Form with fields for name, contact, vehicle number, service type, date, and time.

Admin Dashboard:

Summary statistics + table of bookings with status buttons.

Service Category Page:

List + add new category form.

Login Page:

Simple username/password fields..

1.12 Vehicle Service Booking System (React + Node.js + MongoDB)

Key Features

Customer

- Booking form (name, vehicle number, service type, date, time)
- View booking confirmation

Admin

- Login
- View all bookings
- Approve / Reject / Mark Completed
- Filter bookings by date or status
- Dashboard metrics: ○ Total bookings ○ Pending approvals ○ Completed services ○ Daily bookings

Tech Stack (MERN)

Frontend (React)

- React.js
- Tailwind / CSS
- Axios
- React Router

Backend (Node + Express)

- Node.js
- Express.js • Mongoose

Database

- MongoDB (Atlas or local)

Other

- JWT Authentication
- Bcrypt
- Toast notifications

Folder Structure

```
vehicle-service-app/
├── backend/
│   ├── controllers/
│   ├── routes/
│   ├── models/
│   ├── middleware/
│   ├── config/
│   └── server.js
└── frontend/
    └── src/
        ├── components/
        ├── pages/
        ├── context/
        ├── api/
        └── App.js
```

Database Schema (Simple & Clean)

User (Admin)

{

username: String,

password: String

}

Booking

{

customerName: String, phone: String, vehicleNumber:

String, serviceType: String, date: String, time: String,

status: "Pending" | "Approved" | "Completed" | "Rejected",

createdAt: Date

}

Service Categories

{ name: String, // e.g. "Oil

Change" description: String

}

API Endpoints

Auth

- POST /api/auth/login

Booking

- POST /api/bookings – Create booking
- GET /api/bookings – Get all bookings

- GET /api/bookings/:id – Get a single booking
- PUT /api/bookings/:id/status – Update status
- DELETE /api/bookings/:id – Delete booking

Service Types

- GET /api/services
- POST /api/services

Work Breakdown for Interns Intern

1 – Frontend UI

- Customer booking page
- Admin dashboard UI
- Responsive layout
- Tables, forms, filters

Intern 2 – Frontend Logic

- API integration
- Booking submission
- Status update functionality
- Authentication (admin login)

Intern 3 – Backend Developer

- Booking CRUD API
- Service category API
- Admin auth API
- MongoDB schema & connection

Intern 4 – Deployment & Testing

- Test backend with Postman
- Test UI user flows
- Deploy frontend (Vercel)

- Deploy backend (Render)
- Write documentation

Dashboard Components

- Total Bookings
- Pending
- Approved
- Completed
- Daily booking graph (optional)

Deliverables

- Source Code (GitHub)
- README with setup steps
- Fully working MERN application
- Documentation (functional + technical)
- Deployment link